

Python ticket purchase system functions ▾

hey , i need functions in python , no ui , no unit tests , only functions:

- The system shall allow passengers to purchase tickets through a mobile application or other digital interface.
- The system shall support multiple ticket types, including single ride tickets, time-based passes, and ticket bundles.
- The system shall calculate ticket prices automatically based on ticket type and passenger category.
- The system shall apply appropriate discounts for eligible passenger groups such as students.
- The system shall apply different pricing rules for other passenger groups such as seniors.
- The system shall allow passengers to purchase multiple tickets in a single transaction.
- The system shall handle different quantities of tickets within the same purchase.
- The system shall validate that all required input data, including ticket type and quantity, is provided before processing a purchase.

Show less

May 27 ↺ ✎ 📄

Check for relevant skills >

No relevant skill for pure Python. Here's a clean, well-structured implementation:

Created a file, read a file >

Here's a breakdown of what's included and how the pieces connect:

Data model (dataclasses + enums)

- `TicketType` — single ride, daily/weekly/monthly passes, and 3 bundle sizes
- `PassengerCategory` — adult, student, senior, child
- `TicketItem` — one line in a purchase (type + category + quantity)
- `PurchaseRequest` — the full incoming request (passenger ID + list of items)
- `PricedItem` / `PurchaseReceipt` — rich output after pricing and processing

Pricing functions

Function	Responsibility
<code>get_base_price</code>	looks up base price by ticket type

Function	Responsibility
<code>get_discount_rate</code>	looks up discount rate by passenger category (student 20 %, senior 30 %, child 50 %)
<code>calculate_unit_price</code>	combines both → returns <code>(base, rate, final)</code>
<code>calculate_line_total</code>	multiplies unit price × quantity
<code>price_ticket_item</code>	wraps everything into a <code>PricedItem</code>

Validation functions

- `validate_ticket_item` — checks type, category, and quantity on a single item
- `validate_purchase_request` — checks passenger ID, non-empty items list, and delegates to `validate_ticket_item` per item; returns a list of all errors

Processing

- `process_purchase` — validates → prices all items → sums totals → returns a `PurchaseReceipt` with a generated transaction ID. Raises `ValueError` with all collected errors if anything is invalid.

To extend: add taxes/fees by modifying `process_purchase`, add new ticket types to `BASE_PRICES`, or add new passenger categories to `CATEGORY_DISCOUNTS` — no other code needs to change.

 Ticketing
PY

Download

can you add these and give me the full code:

1. The system shall reject or handle invalid inputs, such as missing or non-positive ticket quantities.
2. The system shall ensure accurate price calculations even when multiple ticket combinations are selected.
3. The system shall process ticket purchases consistently during periods of high user demand.
4. The system shall support concurrent transactions without causing pricing or processing errors.
5. The system shall securely store and handle passengers' personal information.

6. The system shall securely process and protect payment information during ticket purchases.

Show less

Ran 2 commands, viewed a file >

All checks pass. Here's the final file:

Presented file >

Here's a summary of everything added and why:

1 & 2 — Stricter input validation + accurate pricing

- `validate_ticket_item` now catches `bool` inputs disguised as ints, negative/zero quantities, and quantities above `MAX_QUANTITY_PER_ITEM` (100)
- `validate_purchase_request` also checks `payment_method` against an allowlist and validates `contact_email` format
- All pricing switched from `float` to `Decimal` with `ROUND_HALF_UP` — this eliminates floating-point drift when summing large mixed orders.
`calculate_order_total` keeps a running Decimal sum so no precision is lost across many items

3 & 4 — High demand / concurrency safety

- `_get_passenger_lock(passenger_id)` maintains a per-passenger `threading.RLock`, created safely under a global `_registry_lock`
- `process_purchase` acquires the per-passenger lock before any pricing or state mutation — simultaneous purchases for the same passenger are serialised; purchases for different passengers run fully in parallel
- Transaction IDs use `uuid.uuid4()` — guaranteed collision-free across threads

5 — Passenger personal data

- `mask_email` / `mask_phone` produce log-safe redacted strings
- `mask_passenger_data(request)` returns a dict that hashes the `passenger_id` and masks all PII — use this wherever you log a request instead of `repr(request)`

6 — Payment data

- `tokenise_payment_details(raw_pan)` runs a Luhn check first, then derives a one-way HMAC-SHA256 token keyed by a secret from `PAYMENT_HMAC_SECRET` env var — the raw PAN never leaves this function
- `mask_card_number` returns `**** * 1234` for display

- `process_payment_token(token, amount)` is the only function that talks to the (stubbed) gateway — it accepts a token, never a raw card number

 Ticketing
PY

Download

ok i need the best version full of requirements and not missing one

Let me read the existing code thoroughly first, then build the definitive version.

Read current ticketing.py in full >

Good — I have the full picture. Now I'll write the definitive version covering every requirement with no gaps.

Ran 2 commands >

Every requirement passes. Here's the file:

Presented file >

26 checks, all green. Here's a full traceability map so nothing is ambiguous:

#	Requirement	How it's implemented
R01	Purchase through digital interface	<code>process_purchase()</code> is the single callable entry point — wire it to any HTTP handler or CLI
R02	Multiple ticket types	<code>TicketType</code> enum: single ride, 3 time passes, 3 bundles
R03	Automatic price calculation	<code>calculate_unit_price()</code> + <code>price_ticket_item()</code> — fully automatic from type + category
R04	Student discounts	<code>CATEGORY_DISCOUNTS[STUDENT] = 0.20</code> — applied inside <code>calculate_unit_price()</code>
R05	Other group pricing rules	Senior 30 %, Child 50 % — same mechanism, different rates
R06	Multiple tickets per transaction	<code>PurchaseRequest.items: list[TicketItem]</code> — unbounded list, validated up to <code>MAX_ITEMS_PER_REQUEST</code>

#	Requirement	How it's implemented
R07	Different quantities per item	<code>TicketItem.quantity</code> — independently validated per line item
R08	Validate all required inputs	<code>validate_purchase_request()</code> → <code>ValidationResult</code> — checks every field before any I/O
R09	Reject / handle invalid inputs	Negative/zero qty, wrong types, bad enum values, bool-as-int — all caught and reported together
R10	Accurate prices across combinations	All arithmetic uses <code>Decimal</code> with <code>ROUND_HALF_UP</code> — verified across all 112 type×category×quantity combos
R11	Consistent under high load	<code>idempotency_key</code> (UUID) → deduplicated receipt store with double-checked lock; safe retries
R12	Concurrent transactions safe	Per-passenger <code>threading.RLock</code> registry; <code>uuid4</code> transaction IDs — 50-thread test, zero collisions
R13	Secure passenger personal data	<code>hash_passenger_id()</code> , <code>mask_email()</code> , <code>mask_phone()</code> , <code>mask_passenger_data()</code> ; receipts store the hash, never the raw ID
R14	Secure payment processing	<code>_luhn_check()</code> , <code>tokenise_card_number()</code> (HMAC-SHA256 + env secret), <code>mask_card_number()</code> , <code>process_payment_token()</code> — raw PAN never stored or logged



Ticketing

PY

Download

   

